Strumenti di verifica

Il Trono di Cenere — quali strumenti usiamo per controllare il gioco, in ordine di importanza, cosa fanno e come li usiamo. Con l'esito delle verifiche del 29 ottobre 2026 e delle correzioni fatte subito dopo.

In breve

Il gioco resta un sito statico: nessuno di questi strumenti viene caricato dai giocatori. Vivono nella cartella del progetto (`package.json`, `tests/`, `strumenti/`, `.github/`) e servono solo a noi. Una volta installati con `npm install`, il controllo rapido è un solo comando: `npm test`.

La regola che seguo: **prima ciò che protegge i salvataggi e le regole del gioco, poi ciò che trova errori presto, infine ciò che migliora la qualità.** Gli audit con i plugin (Ponytail, Impeccable) restano utili, ma come revisioni periodiche, non come controllo a ogni modifica.

Esito della passata di oggi

Controllo	Risultato	Note
Test delle regole (Node)	**236 / 236**	combattimento 103, magia 80, Sciamano 45, cloud 8
Test nel browser (Playwright)	**105 / 105 combattimento · 18/18 bilanciamento · 20/20 Sciamano**	partita vera, sala di prova, creazione, regolamento, IT/EN
Test di fumo (nuovo)	**0 problemi · 0 avvisi**	5 pagine × PC e Pixel 9a (prima della correzione: 1 problema, 8 avvisi)
Lint per pagina (nuovo)	**0 errori** · 14 avvisi	avvisi minori: variabili assegnate e mai lette
Validazione HTML	**pulito**	struttura, ID duplicati, tag non chiusi
Coerenza documenti e codice (nuovo)	**tutto allineato**	soglie PE, mastini, Ettore, cacce, animali, immagini e audio
Regole Firestore (nuovo)	**non eseguito qui**	l'emulatore non si scarica da questo ambiente: parte su GitHub e sul tuo PC

Problemi trovati e corretti

Corretti il 29 ottobre pomeriggio, dopo la prima passata; tutti i test ripetuti dopo le correzioni.

#	Problema	Dove	Correzione fatta
1	La pagina iniziale chiede *immagini/Trono_di_Cenere.webp* come copertina del video, ma il file non c'è: 404 a ogni apertura e sfondo nero sul telefono finché il video non parte.	index.html riga 221	Creata dal primo fotogramma del video (WebP, 64 KB): la pagina iniziale non è più nera mentre il video si carica.
2	Nei testi del tutorial la chiave *zainoVuoto* compare due volte (italiano e inglese): vince la seconda, «Vuoto.», e «Lo zaino è vuoto.» non si vede mai.	tutorial.html righe 591/600 e 688/697	Tolta la seconda: ora si legge «Lo zaino è vuoto.» / «Your pack is empty.»

#	Problema	Dove	Correzione fatta
3	PE_NEMICI è dichiarata ma non usata e ha i valori di prima (350 e 500): chi la legge, anche un'AI, si fida di numeri sbagliati.	gioco.html riga 6174	Cancellata (i PE veri sono nel campo xp di NEMICI).
4	Bersagli piccoli sul telefono: bandiere della lingua 36×24, «Pagina iniziale» del regolamento 19 px, tendine e oggetti del tutorial 18–20 px, tendine del diario 24 px, scelta facile/difficile 13 px.	index, regolamento, tutorial, gioco, creazione	Bandiere con area di tocco da 44 px (stessa misura a vista), gli altri portati sopra i 32 px.
5	Accessibilità: il diario del tutorial scorre ma non si raggiunge da tastiera; le barre vitali in gioco usano un elenco di definizioni non valido.	tutorial #diario, gioco #vitali	Diario con tabindex e etichetta IT/EN; i separatori delle barre vitali sono ora solo decorativi.
6	Il coniglio dà 20 PE, il documento dice animali normali 50–75.	gioco.html, ANIMALI	Tenuto a 20 e scritto nel documento come eccezione voluta. Se preferisci 50, si cambia un numero.

Gli strumenti, in ordine di importanza

1 Playwright — test nel browser

ESSENZIALE

open source (Microsoft), già nel progetto

COSA FA Apre il gioco in un Chromium vero, senza schermo, e lo usa come un giocatore: clicca, scrive comandi, legge il diario, controlla la console. I dadi si possono truccare, così ogni prova dà sempre lo stesso esito.

NEL GIOCO È la rete principale: 5 file. **combattimento** (105 casi: sala di prova, partita, boss, vie, salvataggi, IT/EN), **bilanciamento** (18), **sciamano** (20) e il nuovo **fumo**, che apre tutte le pagine su PC e in formato Pixel 9a e segnala errori JavaScript, file che non si caricano, pagina più larga dello schermo, bersagli piccoli e accessibilità (con axe-core, punto 7).

COMANDO

```
python3 -m http.server 8765 --bind 127.0.0.1
npm run test:browser
PARTE="2/3" node tests/combattimento.browser.js
```

ESITO DI OGGI Tutto superato (105 / 105 + 18 + 20). Il fumo, dopo le correzioni: nessun problema e nessun avviso.

2 Test delle regole con Node

ESSENZIALE

già nel progetto (tests/.test.js)*

COSA FA Controllano le funzioni pure delle regole (tiri, tecniche, incantesimi, stati, tetti dei boss, sincronizzazione cloud) senza browser, in pochi secondi, con dadi programmati.

NEL GIOCO Si lanciano dopo ogni modifica a js/combattimento.js, magia_combattimento.js, sciamano.js, cloud.js. Gli stessi test girano anche dentro la sala di prova (tutorial.html?prova, pulsante «Esegui i test»).

COMANDO

```
npm run test:regole
```

ESITO DI OGGI 236 / 236 superati.

3 Emulatore Firestore + @firebase/rules-unit-testing

ESSENZIALE

open source (Google), nuovo: firestore.rules, firebase.json, tests/firestore.rules.test.mjs

COSA FA	Avvia sul computer una copia finta di Firestore e prova le regole di sicurezza: chi può leggere e scrivere cosa. Nessun dato vero viene toccato.
NEL GIOCO	Le regole erano solo in un commento di cloud.js. Ora stanno in **firestore.rules**, in una versione più stretta (solo posti 1–3, solo i 4 campi che scrive cloud.js, dati sotto 1 MB, niente cancellazioni dal client). 14 casi: le scritture vere di cloud.js devono passare, gli intrusi no. **Le nuove regole non sono pubblicate**: lo decidi tu dopo che il test passa.
COMANDO	`npm run test:firestore (serve Java)` `npx firebase deploy --only firestore:rules`
ESITO DI OGGI	Non eseguito: questo ambiente non può scaricare l'emulatore. Partirà su GitHub (punto 4) e sul tuo PC.

4 GitHub Actions

IMPORTANTE

servizio gratuito di GitHub, nuovo: .github/workflows/verifiche.yml

COSA FA	A ogni push lancia da solo tutti i controlli su un server di GitHub. Se qualcosa si rompe, il commit prende una X rossa e arriva una mail.
NEL GIOCO	Copre il caso più rischioso per te: una modifica fatta dal telefono, senza poter lanciare i test. Tre gruppi: regole + lint + coerenza, browser, Firestore. Il report del fumo resta scaricabile tra gli artefatti dell'esecuzione.
COMANDO	`scheda «Actions» del repo · pulsante «Run workflow» per lanciarlo a mano`
ESITO DI OGGI	Pronto: si attiva caricando la cartella .github e package.json nel repo.

5 ESLint, una pagina alla volta

IMPORTANTE

open source, nuovo: strumenti/lint_pagine.mjs

COSA FA	Analisi statica: trova variabili mai definite, chiavi duplicate negli oggetti, codice irraggiungibile, funzioni dichiarate due volte. Lo script ricostruisce ogni pagina unendo i file js/ nell'ordine in cui il browser li carica, così trova anche una funzione usata in una pagina che non carica il file giusto.
NEL GIOCO	Utile soprattutto su gioco.html (11.700 righe) e sui dizionari IT/EN, dove una chiave ripetuta cancella in silenzio un testo.
COMANDO	`npm run lint` `node strumenti/lint_pagine.mjs --tutto (anche gli avvisi)`
ESITO DI OGGI	Dopo le correzioni: 0 errori, 14 avvisi minori (variabili assegnate e mai lette).

6 Coerenza documenti e codice

script nostro, nuovo: strumenti/coerenza_docs.mjs — IMPORTANTE

COSA FA	Confronta i numeri decisi in docs/bilanciamento_nemici_oggetti.md con quelli del codice e controlla che ogni immagine e ogni audio citati esistano, con le maiuscole giuste (GitHub Pages le distingue).
NEL GIOCO	Le regole sono cambiate molte volte: il rischio vero è che documento e gioco dicano cose diverse. Oggi controlla soglie PE, PE di mastini, Ettore e cacce, intervallo degli animali, costanti di PE non usate. Si allunga con una riga per ogni nuova decisione.
COMANDO	`npm run coerenza`
ESITO DI OGGI	Dopo le correzioni: tutto allineato.

7 AXE-CORE — ACCESSIBILITÀ

open source (Deque), nuovo, dentro il test di fumo — UTILE

COSA FA	Controlla contrasto dei colori, etichette dei pulsanti, ruoli e struttura, sulla pagina già disegnata (dopo che le traduzioni hanno riempito i testi).
NEL GIOCO	Verifica che la palette Cenere / Tomo Proibito resti leggibile e che il gioco sia usabile anche con lettori di schermo. Segnala solo i problemi gravi e critici.
COMANDO	`parte da solo con node tests/fumo.browser.js`
ESITO DI OGGI	Contrasto a posto su tutte le pagine; dopo le correzioni nessun avviso.

8 HTML-VALIDATE

open source, nuovo: .htmlvalidate.json — UTILE

COSA FA	Controlla la struttura HTML: tag non chiusi, ID ripetuti, elementi annidati male. Le regole sui testi vuoti sono spente, perché i testi li riempie lingua.js dopo il caricamento.
NEL GIOCO	Rapido, gira insieme al lint.
COMANDO	`npx html-validate "*.html"`
ESITO DI OGGI	Pulito.

9 Sala di prova

già nel gioco: tutorial.html?prova — UTILE

COSA FA	Banco di prova del combattimento dentro il gioco: livello, arma, scudo, via, nemico, dadi truccati, test con un pulsante.
NEL GIOCO	Per provare a mano una tecnica nuova senza entrare in partita. I test automatici la usano già.
COMANDO	`apri tutorial.html?prova`
ESITO DI OGGI	Funziona (coperta dai test di Playwright).

10 Audit con plugin: Ponytail e Impeccable

PERIODICO

plugin di Claude Code, già usati il 23 ottobre

COSA FA	Ponytail (/ponytail-audit) legge il codice e propone semplificazioni e correzioni numerate; Impeccable (/impeccable critique) giudica il design con un punteggio e un rilevatore automatico.
NEL GIOCO	Sono revisioni approfondite, non controlli: da rifare dopo un blocco grosso di lavoro (es. dopo i guardiani). I loro report restano in docs/PONYTAIL_AUDIT.md e docs/CRITIQUE.md. Le correzioni del 23 ottobre risultano ancora a posto (punti 1–9 di Ponytail).
COMANDO	`/ponytail-audit · /impeccable critique (in Claude Code)`
ESITO DI OGGI	Non rilanciati oggi: nessun cambiamento grosso dall'ultima volta.

11 Claude in Chrome / Chrome DevTools MCP

SU RICHIESTA

estensione di Claude · server MCP di Google

COSA FA	Aprono il gioco nel tuo Chrome vero (con il tuo accesso Google) e leggono console, richieste di rete e prestazioni.
NEL GIOCO	Servono per ciò che i test non vedono: il sito pubblicato su GitHub Pages, l'accesso Google e la sincronizzazione reale con Firestore, e il Pixel 9a collegato via USB (debug remoto di Chrome). In questa chat c'è Claude in Chrome; DevTools MCP si usa da Claude Code o dall'app desktop.
COMANDO	`su richiesta: «controlla il sito pubblicato»`
ESITO DI OGGI	Non usato oggi: richiede il tuo browser aperto.

Cosa ho scartato e perché

Strumento	Motivo
Supabase	È un'alternativa a Firebase. Avendo già scelto Firebase con accesso Google, due backend sarebbero solo confusione.
oh-my-claudecode	Orchestra più agenti in Claude Code: non verifica niente. Per un progetto di questa dimensione aggiunge complessità senza aggiungere controllo.
Vitest / Jest	I test di Node esistenti fanno già lo stesso lavoro senza dipendenze. Cambiarli non porterebbe nulla.
Lighthouse	Opzionale. Il peso è già sotto controllo: le immagini usate sono WebP sotto 1,2 MB e il video 1,9 MB. I PNG pesanti (6 file, 48 MB) sono originali nel repo, il gioco non li scarica.

Cosa resta da verificare

• **Regole Firestore**: il primo giro dell'emulatore va fatto su GitHub o sul tuo PC (punto 3).

• **Sincronizzazione reale** tra PC e Pixel con il tuo account: i test usano un Firestore finto.

• **Contenuti oltre il piano terra** nel test di fumo: oggi apre la partita e prova un comando; un percorso guidato più lungo (cucina, Guendalina, cantina, riposo) è il prossimo passo naturale.

Uso consigliato

Quando	Cosa lanciare
Dopo ogni modifica	`npm test` (regole, lint, coerenza: meno di un minuto)
Prima di caricare su GitHub	`npm run test:browser` (o lasciarlo fare a GitHub Actions)

Quando	Cosa lanciare
Quando cambiano le regole Firestore	`npm run test:firestore`, poi pubblicare
Dopo un blocco grosso di lavoro	Ponytail e Impeccable
Prima di dire «fatto» su mobile	Claude in Chrome sul sito pubblicato, o il Pixel via debug remoto